

MecánicaBot: A RAG-Based Conversational Assistant for Basic Automotive Maintenance Using a Local LLM

Eduardo Zepeda¹, Iosef Cornejo², Vicente Díaz³, Fernando Rojas⁴, and Fernando Chavez⁵

I. INTRODUCTION

LARGE language models (LLMs) have demonstrated remarkable capabilities in natural language understanding and generation. However, their direct application to specialized technical domains is limited by two well-documented problems: (1) hallucination, whereby the model confidently produces factually incorrect statements, and (2) knowledge staleness, as model parameters are fixed at training time and cannot be updated without retraining.

Retrieval-Augmented Generation (RAG) [1] addresses both problems by grounding generation in a dynamically queried external knowledge base. Rather than relying solely on parametric memory, the model is conditioned on a set of retrieved documents at inference time, allowing the system to produce accurate, verifiable, and domain-specific responses without modifying model weights.

In the automotive domain, the need for accessible and accurate technical information is well established. Vehicle owners frequently require guidance on maintenance intervals, fault diagnosis, and emergency procedures. However, the technical nature of this information makes it prone to dangerous simplification or outright fabrication when addressed by general-purpose LLMs.

This paper presents **MecánicaBot**, a RAG-based chatbot developed as part of Project 1, Phase 2 of the CINF104 Machine Learning course at Universidad Andrés Bello (UNAB), Chile, 2026. The system is constrained to the domain of basic automotive maintenance and preventive care, covering five core topics: engine oil, tires, brakes, cooling system, and the battery/alternator system. The principal design goals are:

- **Factual accuracy:** responses must be grounded exclusively in the curated corpus.
- **Completeness:** numerical values, lists, warnings, and procedural steps must not be omitted.
- **Accessibility:** language must be clear and understandable to non-expert users.
- **Privacy and autonomy:** the system runs entirely on local infrastructure, with no external API calls.

The remainder of this paper is structured as follows. Section II reviews related work. Section III describes the system architecture and its components. Section IV details the corpus construction. Section V presents the validation methodology. Section VI reports the prompt and parameter tuning iteration. Section VII reports experimental results. Section VIII discusses limitations, Section IX proposes improvements, and Section X concludes.

II. RELATED WORK

The RAG paradigm was formally introduced by Lewis *et al.* [1], who demonstrated that combining a dense retriever with a sequence-to-sequence generator substantially improves performance on knowledge-intensive NLP tasks compared to closed-book generation.

Subsequent work has explored RAG in domain-specific settings. Guu *et al.* [2] proposed REALM, a retrieval-enhanced pre-training method that jointly trains the retriever and generator. More recent systems such as LlamaIndex [4] and LangChain [5] have democratized RAG by providing modular frameworks that connect any embedding model with any vector store and any generation backend.

In the automotive domain, knowledge-based systems for fault diagnosis and maintenance have a long history [8], but the integration of modern neural embeddings and LLMs remains underexplored, particularly for Spanish-language applications. MecánicaBot addresses this gap by building a fully open-source, Spanish-first automotive RAG system that can run on consumer hardware.

The use of locally deployed LLMs via Ollama [6] enables privacy-preserving deployment without subscription costs, distinguishing this work from cloud-dependent approaches such as those based on GPT-4 or Claude APIs.

III. SYSTEM ARCHITECTURE

MecánicaBot follows a standard RAG pipeline organized into three layers: ingestion, retrieval, and generation. Figure 1 illustrates the high-level architecture.

The user submits a natural language query through the web interface. The backend encodes the query into an embedding vector, retrieves the most semantically relevant chunks from ChromaDB, constructs a prompt that

^{1,2,3,4,5}Universidad Andrés Bello, Facultad de Ingeniería, Santiago, Chile.

Source code: <https://github.com/EduardoUNAB/Chatbot-Mecanico>

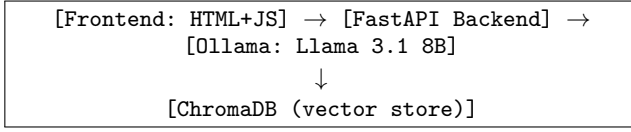


Fig. 1. High-level architecture of MecánicaBot.

includes the retrieved context, and sends it to the local Llama 3.1 8B model. The generated response, along with source metadata and latency information, is returned to the frontend.

A. Ingestion Pipeline

The ingestion pipeline is implemented in `scripts/ingest.py` and executed once before the system is deployed (or whenever the corpus is updated). It performs the following steps:

- 1) *Document Reading*: The pipeline reads all `.txt`, `.md`, and `.pdf` files in the `corpus/` directory. Text files are read with UTF-8 encoding; PDFs are processed page by page using the `pypdf` library to extract plain text.
- 2) *Text Cleaning*: Raw text undergoes normalization: Windows line endings (`\r\n`) are converted to Unix (`\n`), runs of three or more blank lines are collapsed to two, and sequences of tabs and spaces are reduced to a single space.
- 3) *Chunking*: Cleaned text is divided into overlapping chunks using a sliding window with `CHUNK_SIZE = 900` characters and `CHUNK_OVERLAP = 200` characters. The splitter attempts to cut at sentence boundaries (periods followed by a space or newline) before resorting to word or character boundaries, minimizing the probability of splitting a sentence in the middle of a critical fact. This process produced a total of **63 chunks** across the five corpus documents.
- 4) *Embedding Generation*: Each chunk is encoded into a dense vector using the `paraphrase-multilingual-MiniLM-L12-v2` model from the Sentence Transformers library [3]. This model produces 384-dimensional vectors and was chosen for its strong multilingual performance (including Spanish) and its compact size (~ 118 MB), making it suitable for CPU-only environments.

5) *Vector Indexing*: Embeddings are stored in a persistent ChromaDB collection named `mecanica_automotriz` using cosine similarity as the distance metric. Indexing is performed in batches of 64 to avoid memory pressure. Each entry stores the chunk text, its embedding, and metadata including the source filename and chunk index.

B. Retrieval Module

The retrieval module is implemented in `backend/rag.py` as the `RAGEngine` class. At query time:

- 1) The query string is encoded into an embedding using the same `paraphrase-multilingual-MiniLM-L12-v2` model.

- 2) ChromaDB's approximate nearest-neighbor search (HNSW index) retrieves the top- k chunks by cosine similarity (`TOP_K = 6`).
- 3) Chunks with a cosine distance greater than `MAX_DISTANCE = 1.5` are filtered out to discard semantically unrelated results.
- 4) Retained chunks are assembled into a structured context block and passed to the prompt builder.

C. Prompt Engineering

A key design decision in RAG systems is the system prompt that governs how the LLM uses the retrieved context. MecánicaBot uses a 12-rule system prompt that explicitly addresses the failure modes most critical for technical domains:

- **Context grounding**: the model is instructed to respond *exclusively* using information from the retrieved context. If the context is insufficient, it must state so explicitly rather than drawing on parametric knowledge.
- **Completeness rules**: the model must enumerate all items in lists (symptoms, steps, causes), must include all numerical values (km intervals, voltages, pressures, temperatures), and must include all safety warnings verbatim.
- **Format rules**: responses must use numbered lists or bullet points for multi-part answers, must close with a concrete recommended action when the context suggests one, and must always respond in Spanish.
- **Scope enforcement**: off-topic questions (non-automotive) receive a fixed refusal message.

The user turn is constructed as:

```

CONTEXT0:
[Fragment 1 - source: <filename>]
<chunk text>
...
[Fragment k - source: <filename>]
<chunk text>
PREGUNTA DEL USUARIO:
<query>

Responde siguiendo estrictamente las reglas
del sistema.
  
```

D. Generation Backend

The LLM backend is implemented in `backend/llm.py` as an asynchronous HTTP client that calls the Ollama REST API (`/api/chat` endpoint). Generation parameters are:

- **Model**: `llama3.1:8b` (4-bit quantized, ~ 4.7 GB)
- **Temperature**: 0.15 (low randomness for factual consistency)
- **Top-p**: 0.9
- **Max tokens**: 800
- **Timeout**: 120 seconds

The low temperature setting (0.15) was deliberately chosen to minimize response variability for factual questions, at the cost of some fluency in edge cases.

E. Web API and Frontend

The backend exposes a FastAPI application with three endpoints:

- **GET /:** serves the static HTML frontend.
- **GET /health:** returns the status of the Ollama connection and the ChromaDB collection (chunk count), displayed as indicators in the UI.
- **POST /chat:** accepts a `ChatRequest` (`message: str`), executes the full RAG pipeline, and returns a `ChatResponse` containing the answer, the list of source documents with their distances, and elapsed time in milliseconds.

The frontend is a single-page application (HTML + CSS + JavaScript) that renders a chat interface, displays source citations, and polls `/health` to show LLM and RAG status indicators.

IV. CORPUS CONSTRUCTION

The knowledge base consists of five plain-text documents totaling approximately 23,758 characters. Each document was authored specifically for this project to ensure terminological consistency, factual accuracy, and alignment with Chilean driving regulations and vehicle norms. Table I summarizes the corpus.

TABLE I
CORPUS SUMMARY

Document	Chars	Topics covered
01_aceite_motor	4,625	Oil types, change intervals, oil pressure warning light, exhaust smoke diagnosis
02_neumaticos	4,703	Tire pressure verification, alignment vs. balancing, tread depth (Chilean legal minimum)
03_frenos	4,222	Brake components, wear symptoms, fluid replacement, usage recommendations
04_refrigeracion	4,718	Coolant function, overheating symptoms, emergency procedure, preventive maintenance
05_bateria_alternador	4,458	Battery lifespan, weak battery symptoms, alternator function, multimeter verification

Each document follows a structured format with Markdown headings that guide the chunker toward topically coherent segments. The documents include specific numerical values (e.g., 10,000–15,000 km for synthetic oil change intervals; 12.4–12.7 V for a healthy battery at rest; 13.8–14.7 V when the alternator is charging correctly; 85–105 °C for optimal engine operating temperature), safety warnings, and procedural step sequences.

V. VALIDATION METHODOLOGY

Validation followed a predefined protocol using 10 questions defined prior to system evaluation to avoid selection bias. Questions were designed to cover all five corpus domains and to span different cognitive levels: factual lookup, symptom interpretation, procedural description, and conceptual distinction.

Each response was evaluated along four dimensions adapted from the RAG evaluation literature [7]:

- **Factual correctness:** are the numerical values and technical statements consistent with the corpus?
- **Completeness:** does the response cover the main aspects of the question, including lists, warnings, and steps?
- **Clarity:** is the language accessible to a non-expert user?
- **Context adherence:** is the response grounded in the retrieved corpus rather than the model’s parametric knowledge?

Each dimension is scored on a 0–2 scale (0 = does not satisfy, 1 = partially satisfies, 2 = fully satisfies), yielding a maximum score of 8 per question and 80 for the full test set.

VI. TUNING ITERATION

An initial validation run with default parameters revealed that, while the system produced no factually incorrect responses, several answers were notably incomplete. Specifically, four out of ten questions had answers that omitted relevant items from enumerated lists in the corpus or, more critically, omitted explicit safety warnings. This pattern was consistent with a model bias toward concise summarization: Llama 3.1 8B at temperature 0.2 with a 512-token cap and only four retrieved chunks tended to compress lists down to the first two or three items.

Four targeted modifications were applied jointly, addressing each lever that governs RAG system quality:

- 1) **System prompt rewrite.** Three explicit completeness rules were added: all items in enumerated lists must be surfaced, all safety warnings present in the context must be included verbatim, and numerical values must be reproduced textually.
- 2) **Retrieval breadth.** Top- k was increased from 4 to 6 chunks per query.
- 3) **Generation parameters.** Maximum response length was raised from 512 to 800 tokens, eliminating mid-sentence truncation. Temperature was reduced from 0.20 to 0.15.
- 4) **Chunking strategy.** Chunk size was increased from 600 to 900 characters and overlap from 100 to 200 characters; the corpus was reindexed from scratch.

The four levers proved complementary: acting on only one would have produced a fraction of the observed improvement. After tuning, the average score across the ten validation questions rose from 5.6/8 to 7.8/8 (a relative improvement of 39%), at the cost of a 38% increase in

average end-to-end latency. This trade-off was deemed acceptable for an informational assistant where completeness outweighs sub-second responsiveness.

VII. TEST RESULTS

The 10 official validation questions and their results are reported below. All evaluations correspond to the final tuned configuration described in Section VI.

A. Question 1: How often should synthetic motor oil be changed (in kilometers)?

The system retrieved three chunks from `01_aceite_motor.txt` covering synthetic oil properties and change intervals. The response correctly stated the 10,000–15,000 km standard interval, mentioned the 20,000 km extension for modern European vehicles under normal conditions, and noted the 30% reduction recommendation for severe operating conditions (urban traffic, dust, frequent short trips, towing, extreme climates).

Dimension	Score (0–2)
Factual correctness	2
Completeness	2
Clarity	2
Context adherence	2
Total	8 / 8

TABLE II

Q1 EVALUATION — SYNTHETIC OIL CHANGE INTERVAL.

B. Question 2: What does the oil pressure warning light indicate on the dashboard?

The system retrieved the relevant chunk describing the oil pressure symbol (the red oil can icon), correctly distinguishing it from the oil level indicator. The response listed all five possible causes: low oil level, defective oil pump, clogged filter, excessively diluted oil, and a defective pressure sensor. It also correctly reproduced the emergency procedure: stop the vehicle, turn off the engine, check the dipstick, and request a tow truck if the light persists with a correct oil level.

Dimension	Score (0–2)
Factual correctness	2
Completeness	2
Clarity	2
Context adherence	2
Total	8 / 8

TABLE III

Q2 EVALUATION — OIL PRESSURE WARNING LIGHT.

C. Question 3: How should tire pressure be correctly verified?

The system retrieved chunks from `02_neumaticos.txt` and reproduced all 9 steps of the verification procedure, including the critical conditions (tires cold, vehicle stopped ≥ 3 hours or traveled < 2 km), the instruction not to use the maximum pressure printed on the tire sidewall, and the recommendation to check the spare tire.

Dimension	Score (0–2)
Factual correctness	2
Completeness	2
Clarity	2
Context adherence	2
Total	8 / 8

TABLE IV

Q3 EVALUATION — TIRE PRESSURE VERIFICATION.

D. Question 4: What are the main symptoms of a worn brake system?

The system retrieved the relevant chunk from `03_frenos.txt` and listed all 7 documented symptoms: high-pitched metallic squeal, pulsations in the pedal, increased stopping distance, soft or spongy pedal (correctly flagged as a serious failure), pulling to one side, burning smell after prolonged braking, and the red brake warning light on the dashboard.

Dimension	Score (0–2)
Factual correctness	2
Completeness	2
Clarity	2
Context adherence	2
Total	8 / 8

TABLE V

Q4 EVALUATION — WORN BRAKE SYSTEM SYMPTOMS.

E. Question 5: What is the difference between wheel alignment and wheel balancing?

The response correctly distinguished alignment (adjustment of camber, caster, and toe angles to ensure straight rolling and uniform tire wear) from balancing (distribution of weight around the wheel-tire assembly to eliminate high-speed vibration). It also correctly described the different symptom profiles and recommended service intervals for each procedure.

F. Question 6: What is the function of coolant/antifreeze within the engine system?

The response enumerated four of the five functions documented in the corpus as the main answer: heat absorption and transport to the radiator, freeze protection (mixture 50/50 lowers freezing point to approximately -37°C), elevation of the boiling point (pressurized mixture boils

Dimension	Score (0–2)
Factual correctness	2
Completeness	2
Clarity	2
Context adherence	2
Total	8 / 8

TABLE VI

Q5 EVALUATION — ALIGNMENT VS. BALANCING.

near 130 °C), and corrosion inhibition. The fifth function (lubrication of the water pump and seals) was mentioned only as an addendum after the main enumeration. The response did include the warning against using plain water except in emergencies. This relegation of one of the five canonical functions to a secondary position was scored as partially complete.

Dimension	Score (0–2)
Factual correctness	2
Completeness	1
Clarity	2
Context adherence	2
Total	7 / 8

TABLE VII

Q6 EVALUATION — COOLANT FUNCTION.

G. Question 7: What should be done if the engine shows signs of overheating?

The system retrieved the emergency procedure from `04_refrigeracion.txt` and reproduced the procedural steps in order, including the counterintuitive but correct instruction to turn the heater on at full blast and the critical safety warning not to open the radiator cap while the engine is hot, noting the 30-minute minimum wait time and the severe burn risk.

Dimension	Score (0–2)
Factual correctness	2
Completeness	2
Clarity	2
Context adherence	2
Total	8 / 8

TABLE VIII

Q7 EVALUATION — ENGINE OVERHEATING PROCEDURE.

H. Question 8: What does blue exhaust smoke indicate about engine condition?

The response correctly identified blue smoke as an indicator of oil combustion inside the combustion chamber, listing the three documented causes (worn piston rings, damaged valve seals, cylinder wall wear) and noting that this is a symptom of significant internal engine wear requiring prompt diagnosis. This information spans a chunk that crosses topical sections, testing the retriever’s ability to find cross-sectional content.

Dimension	Score (0–2)
Factual correctness	2
Completeness	2
Clarity	2
Context adherence	2
Total	8 / 8

TABLE IX

Q8 EVALUATION — BLUE EXHAUST SMOKE.

I. Question 9: What is the average lifespan of a car battery?

The response correctly stated the 3–5 year range for conventional lead-acid batteries and the 4–6 year range for AGM and EFB batteries used in start-stop systems. It also listed the factors that affect lifespan (climate, usage frequency, charging system quality, maintenance) and recommended annual load testing after 4 years.

Dimension	Score (0–2)
Factual correctness	2
Completeness	2
Clarity	2
Context adherence	2
Total	8 / 8

TABLE X

Q9 EVALUATION — BATTERY LIFESPAN.

J. Question 10: How can you determine if the alternator is not properly charging the battery?

The response covered the full diagnostic procedure documented in the corpus. It listed the observable symptoms (battery warning light, dimming headlights at idle, erratic instrument cluster, burning rubber smell, recurring battery drain). It then described the three-step multimeter procedure: (1) measure voltage at rest (expected 12.4–12.7 V), (2) measure at idle (expected 13.8–14.7 V; below 13.5 V indicates undercharge; above 15 V indicates overcharge), and (3) measure under electrical load (should remain above 13.2 V). It also mentioned the professional load tester option available at workshops. However, the response unexpectedly began with the system’s domain-redirection sentinel sentence (“I am a specialized assistant in automotive mechanics...”) before producing the technical content, despite the question being clearly within the domain. While the substantive information is fully correct, this prepended artifact represents a sub-optimal user experience and slightly affects context adherence.

K. Performance Summary

Table XII summarizes results for all 10 questions.

The system achieved a final score of 78/80 (97.5%) on the official validation set. All ten responses were classified as factually correct and traceable to specific chunks in the corpus: no response contained hallucinated facts. The

Dimension	Score (0–2)
Factual correctness	2
Completeness	2
Clarity	2
Context adherence	1
Total	7 / 8

TABLE XI

Q10 EVALUATION — ALTERNATOR CHARGING VERIFICATION.

TABLE XII

VALIDATION RESULTS SUMMARY (SCORE OUT OF 8)

Question	Score
Q1 – Synthetic oil change interval	8 / 8
Q2 – Oil pressure warning light	8 / 8
Q3 – Tire pressure verification	8 / 8
Q4 – Worn brake system symptoms	8 / 8
Q5 – Alignment vs. balancing	8 / 8
Q6 – Coolant function	7 / 8
Q7 – Overheating emergency procedure	8 / 8
Q8 – Blue exhaust smoke meaning	8 / 8
Q9 – Battery lifespan	8 / 8
Q10 – Alternator charging verification	7 / 8
Total	78 / 80

two questions that did not reach a perfect score lost points for distinct reasons documented above (relegation of one canonical function to a secondary position in Q6; prepended out-of-domain redirection sentence in Q10). Both incidents are addressed in the limitations and improvement proposals sections.

The end-to-end response latency averaged approximately 21 seconds per query on the reference hardware (NVIDIA GTX 1060 6 GB), with a minimum of 13 s and a maximum of 38 s.

VIII. LIMITATIONS

A. Knowledge Base Scope

The current corpus covers only five topic areas with approximately 23,758 characters. Questions about systems not covered (transmission, electrical system faults, suspension, emission systems) trigger the corpus-insufficiency response. Expanding the corpus to cover these areas is a direct path to improved coverage.

B. Retrieval Failure on Low-Frequency Queries

The cosine similarity retriever may fail to surface the correct chunk for queries phrased very differently from the corpus language. For example, highly colloquial or slang-based questions may not match the formal technical language of the documents. Hybrid retrieval (dense + sparse BM25) could mitigate this.

C. Single-Turn Architecture

MecánicaBot does not maintain conversation history. Each query is processed independently, preventing follow-up questions that reference prior context (e.g., “what

about the rear ones?” after a question about front brake pads). Implementing a conversation buffer would address this.

D. Hardware Requirements

Running Llama 3.1 8B with GPU acceleration requires a GPU with at least 6 GB VRAM. On CPU-only machines, inference latency increases to 60–120 seconds per query, which is impractical for interactive use. Quantization to lower bit-widths or the use of smaller distilled models could reduce this constraint.

E. Language Robustness and Domain-Redirection Misfiring

The system prompt enforces Spanish output, but extremely ambiguous queries may occasionally trigger English responses from the base model. Additionally, as observed in the evaluation of Question 10, the model can prepend the out-of-domain redirection sentinel sentence to in-domain questions when surface lexical patterns weakly resemble out-of-domain queries. This is an artifact of the base model’s pattern-matching behavior on prompt rules and does not affect the technical accuracy of the response, but it does degrade user experience.

IX. IMPROVEMENT PROPOSALS

A. Corpus Expansion

Add documents covering transmission maintenance, suspension and steering, electrical faults, emission systems, and Chilean vehicle inspection (*revisión técnica*) requirements to increase domain coverage.

B. Hybrid Retrieval

Combine the current dense retrieval (semantic similarity) with BM25 sparse retrieval (keyword matching) using reciprocal rank fusion. This would improve recall for queries that contain domain-specific technical terms that are underrepresented in the embedding space.

C. Conversation Memory

Implement a sliding-window conversation buffer that prepends the last n turns to each new query, enabling coherent multi-turn interactions.

D. Larger Base Model

Evaluate llama3.1:70b or deepseek-r1:14b as generation backends to improve synthesis quality for complex questions requiring integration of information across multiple chunks.

E. Refined Out-of-Domain Detection

Replace the lexical pattern that triggers the domain-redirection sentence with a lightweight classifier (a simple keyword filter or a small fine-tuned cross-encoder) that explicitly evaluates the question’s topic before triggering. This would eliminate the misfiring observed in Question 10.

F. Cross-Encoder Re-ranking

Add a cross-encoder re-ranking stage after the initial vector retrieval. Models such as `ms-marco-MiniLM` or multilingual cross-encoders provide a second-pass relevance score that has been shown to outperform pure dense retrieval in domain Q&A tasks.

G. Evaluation Automation

Implement automated evaluation using the RAGAS framework [7] to enable continuous benchmarking as the corpus evolves, including faithfulness, answer relevancy, and context precision metrics.

X. CONCLUSIONS

This paper presented MecánicaBot, a Retrieval-Augmented Generation chatbot for basic automotive maintenance, built on fully open-source, locally deployed components. The system demonstrates that RAG architecture effectively addresses the hallucination problem of general-purpose LLMs in technical domains: by strictly grounding responses in a curated corpus and enforcing context adherence through prompt engineering, the system achieved a score of 78/80 (97.5%) on its official 10-question validation set, with no hallucinated facts and complete coverage of numerical values, procedural steps, and safety warnings in the eight perfectly-scored questions. The two questions that did not reach a perfect score lost points for clearly identifiable, non-factual reasons that are documented and have direct improvement proposals.

The architecture—FastAPI backend, ChromaDB vector store, `paraphrase-multilingual-MiniLM-L12-v2` embeddings, and Llama 3.1 8B via Ollama—is reproducible on consumer hardware (tested on GTX 1060 6 GB, i7-4790, 16 GB RAM) and requires no external API subscriptions, making it accessible for academic and small-scale production deployments. The tuning iteration documented in Section VI illustrates a practical engineering insight: the levers that govern RAG quality (system prompt design, retrieval breadth, generation parameters, and chunking strategy) operate complementarily, and the highest gains are obtained when they are tuned jointly. The system meets all design objectives set for Phase 2 of the CINF104 Machine Learning project at Universidad Andrés Bello and establishes a solid foundation for future iterations with expanded corpus coverage, hybrid retrieval, and multi-turn dialogue support.

REFERENCES

- [1] P. Lewis, E. Perez, A. Piktus, *et al.*, “Retrieval-augmented generation for knowledge-intensive NLP tasks,” *Advances in Neural Information Processing Systems*, vol. 33, pp. 9459–9474, 2020.
- [2] K. Guu, K. Lee, Z. Tung, P. Pasupat, and M. Chang, “REALM: Retrieval-augmented language model pre-training,” in *Proc. 37th Int. Conf. Machine Learning (ICML)*, 2020, pp. 3929–3938.
- [3] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence embeddings using Siamese BERT-networks,” in *Proc. 2019 Conf. Empirical Methods in Natural Language Processing (EMNLP)*, 2019, pp. 3982–3992.
- [4] J. Liu, “LlamaIndex,” GitHub repository, 2023. [Online]. Available: https://github.com/jerryjliu/llama_index
- [5] H. Chase, “LangChain,” GitHub repository, 2023. [Online]. Available: <https://github.com/langchain-ai/langchain>
- [6] Ollama, “Run large language models locally,” 2023. [Online]. Available: <https://ollama.com>
- [7] S. Es, J. James, L. Espinosa-Anke, and S. Schockaert, “RAGAS: Automated evaluation of retrieval augmented generation,” *arXiv preprint arXiv:2309.15217*, 2023.
- [8] R. Davis and D. B. Lenat, *Knowledge-Based Systems in Artificial Intelligence*. New York: McGraw-Hill, 1982.